Diffusion models

Métodos Generativos, curso 2026-2027

Guillermo Iglesias, guillermo.iglesias@upm.es
Edgar Talavera Muñoz, e.talavera@upm.es

7 de noviembre de 2026

Escuela Técnica Superior de Ingeniería de Sistemas Informáticos | UPM



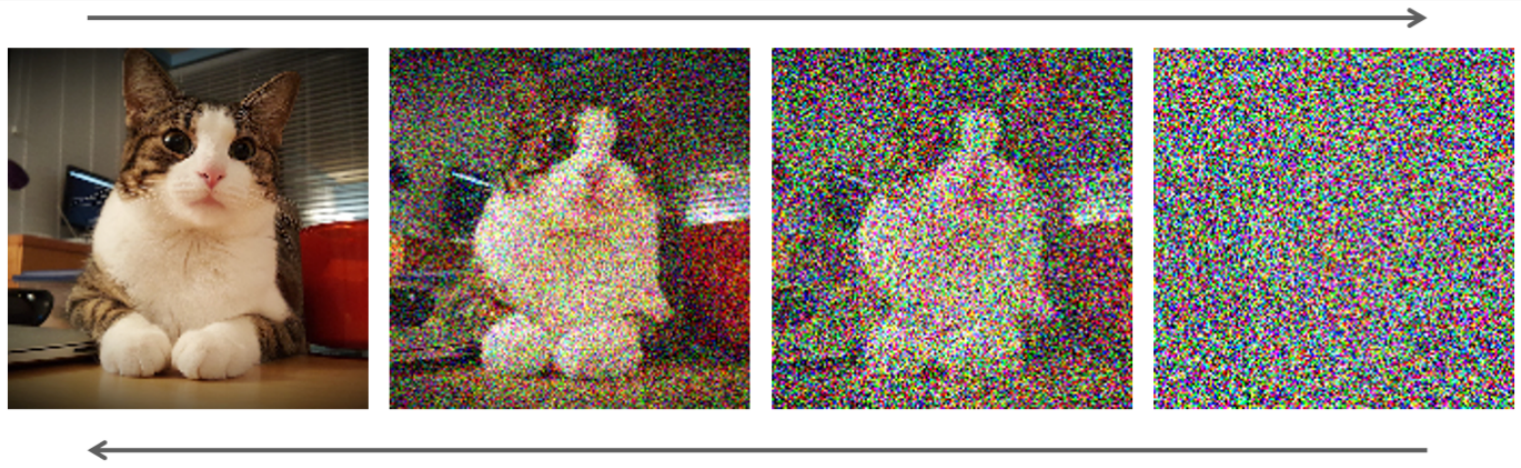
Introducción e intuición previa

¿Qué es un modelo de difusion?

Los modelos de difusión o diffusion models, propuestos en el año 2015 [1], tienen su base en la termodinámica.

Se encuentran dentro de los modelos de variables latentes, esto es, generan nuevos datos usando un espacio latente, como los Autoencoder (AE), Variational Autoencoder (VAE) o Generative Adversarial Network (GAN).

Su funcionamiento se basa en aplicar denoising a un dato de manera iterativa.



[2]

Diffusion models en la actualidad

Los diffusion models suponen la tecnología para generación de imágenes más avanzada actualmente.

Aplicaciones como Stable Diffusion utilizan variaciones de la arquitectura de los modelos de difusión.

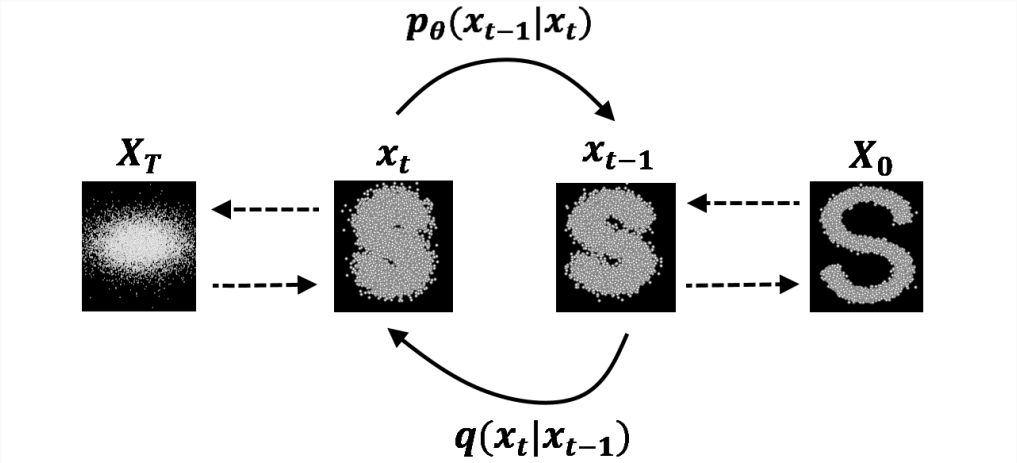
Respecto a arquitecturas anteriores como las GAN, los modelos de difusión solucionan problemas como el colapso modal o gradient vanishing, al mismo tiempo que generan imágenes más definidas y diversas.

Cadenas de Markov

La arquitectura básica de un diffusion model se basa en las cadenas de Markov, que es un modelo probabilístico de cambio de estados.

La principal idea es a partir de una imagen[1] generar versiones más ruidosas de la misma de manera iterativa y encadenada.

Finalmente, una red de neuronas aprenderá a realizar el proceso inverso, eliminar el ruido de las imágenes.



[3]

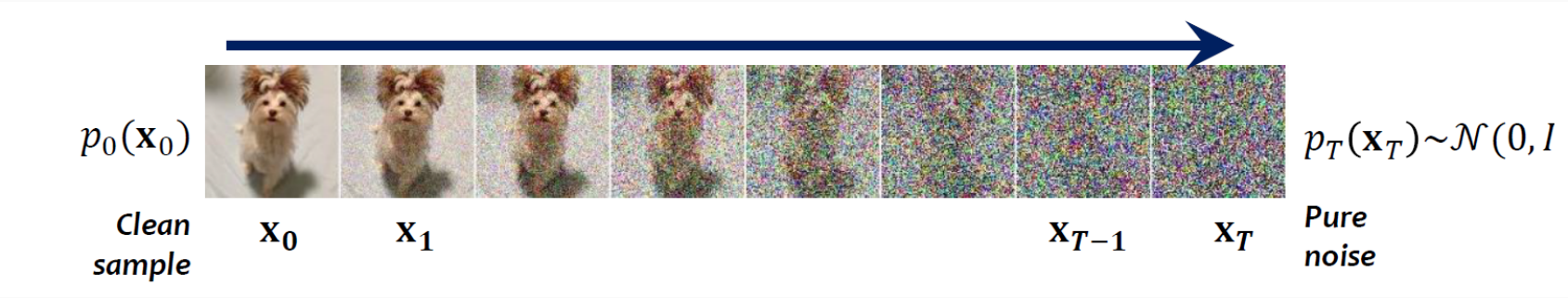
[1]Aunque los diffusion models se pueden aplicar a distintas fuentes de datos, por simplicidad de hablará sólo de imágenes.

Sampleo de datos (difusión directa e inversa)

Difusión directa

Como se ha explicado anteriormente, el funcionamiento de los diffusion models se basa en añadir iterativamente ruido a la imagen de entrada.

Cada instante $t+1$ es generado a partir del instante t añadiendo ruido gaussiano a la imagen.



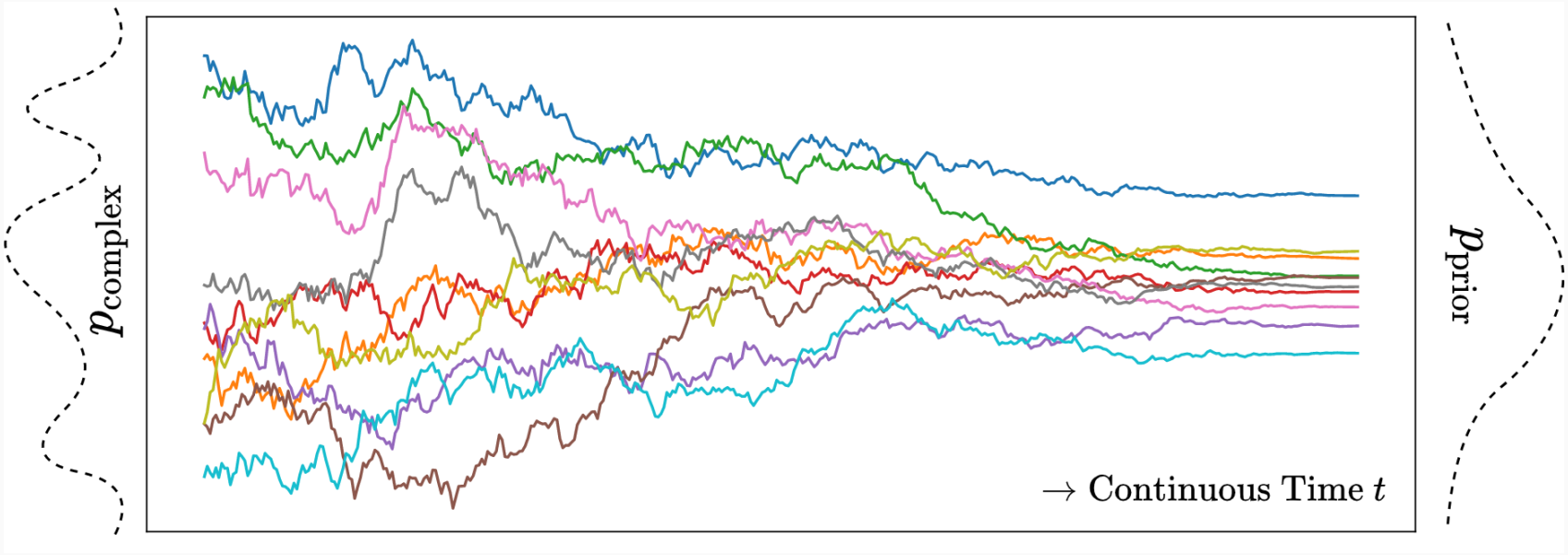
[2]

Cabe destacar que la magnitud del ruido generado depende el instante t en el que se encuentre ese ejemplo.

Difusión directa

A través de la introducción de ruido en las imágenes, se consigue cambiar la distribución de datos de los ejemplos del dataset. Cada eslabón de la cadena de Markov de nuestro modelo corresponde a una distribución distinta.

Para el instante T la distribución seguida por los datos será una gaussiana.



[4]

¿Por qué llegar hasta imágenes completamente ruidosas?

La idea para generar nuevas imágenes es generar ruido aleatorio y realizar le difusión inversa.

De esta manera es posible generar nuevos ejemplos aleatoriamente, ya que la distribución de datos del instante T es conocida y la del instante 0 se calcula a través del proceso de difusión inversa.

Difusión directa: computación

A la hora de introducir ruido de manera iterativa en las imágenes que se están procesando hay distintas alternativas.

Para cada imagen generada en un instante t esta se puede definir como[2]:

$$x_t = \underbrace{\sqrt{\alpha_t} x_{t-1}}_{\text{Media (determinista)}} + \underbrace{\sqrt{1-\alpha_t}\epsilon_{t-1}}_{\text{Varianza} \times \text{Ruido}} \tag{1}$$

donde α es el nivel de imagen original que se mantiene, que se calcula para cada instante t dependiendo de un planificador llamado "variance scheduler". Y ϵ_{t-1} es el ruido de cada instante anterior tal que $\epsilon \sim \mathcal{N}(0, 1)$.

[2]Lo normal es encontrar la fórmula definida con el diffusion rate $\beta = 1 - \alpha$

Difusión directa: computación

Calcular el nivel de ruido iterativamente paso a paso es muy costoso computacionalmente. Por lo que si definimos la fórmula iterativa de la siguiente manera:

$$x_t = \sqrt{\alpha_t} \underbrace{\left(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}} \epsilon_{t-2} \right)}_{x_{t-1}} + \sqrt{1-\alpha_t} \epsilon_{t-1} \tag{2}$$

$$x_t = \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{\alpha_t (1-\alpha_{t-1})} \epsilon_{t-2} + \sqrt{1-\alpha_t} \epsilon_{t-1} \tag{3}$$

Cabe resaltar que los dos ruidos $\epsilon_{t-1}, \epsilon_{t-2} \sim \mathcal{N}(0,1)$ son independientes y siguen la misma distribución. Con ello podemos obtener $\bar{\epsilon}$ combinándolos y con ello:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_t \alpha_{t-1}} \bar{\epsilon} \tag{4}$$

Difusión directa: el truco final

Si aplicamos esta misma propiedad de manera recursiva hasta llegar a la imagen original x_0, los términos α se van multiplicando.

Podemos definir $\bar{\alpha}_t$ como el producto acumulado de todos los instantes:

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s \tag{5}$$

Lo que nos permite definir cualquier instante t en un solo paso directamente desde x_0:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \tag{6}$$

Esta propiedad permite que el entrenamiento sea altamente paralelizable, calculando cualquier estado x_t en tiempo $\mathcal{O}(1)$.

Planificadores de ruido (Noise Schedulers)

La forma en la que evoluciona el α_t durante los pasos de tiempo es crucial y viene definida por el Noise Scheduler.

- **Planificador Lineal (DDPM original):** Incrementa α_t de forma lineal. Esto destruye demasiada información demasiado rápido.
- **Planificador Coseno [5]:** Evoluciona las $\bar{\alpha}_t$ siguiendo una curva trigonométrica. Esto hace que la transición sea más suave.



[5]

Difusión inversa

La idea del proceso de inverse diffusion es volver al ejemplo original partiendo del ruidoso.

Para ello se realiza iterativamente la operación de pasar del instante t al instante $t-1$.

Para generar la imagen del instante $t-1$ son necesarios:

- La imagen de entrada del instante t.
- El ruido de dicha imagen calculado con nuestra red de neuronas.
- Un algoritmo para sustraer el ruido de la imagen.

Matemáticamente, el proceso se define de la manera siguiente:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \tag{7}$$

Difusión inversa: ¿Qué debe aprender la red?

Durante el proceso de difusión directa, hemos añadido un ruido conocido $\epsilon \sim \mathcal{N}(0, 1)$ para pasar de x_0 a x_t.

Para revertir el proceso, necesitamos una red neuronal (más adelante se explica en profundidad) parametrizada por θ.

Inicialmente, se intentó que la red predijera la imagen original x_0 o la imagen anterior x_{t-1}. Sin embargo, Ho et al. (2020) [6] demostraron que lo más eficiente matemáticamente es que la red intente predecir el ruido exacto que se añadió en el instante t.

Definimos nuestra red como: $\epsilon_\theta(x_t, t)$

Entrenamiento y Función de Pérdida

Para entrenar la red, el proceso es sorprendentemente sencillo. Tomamos una imagen original x_0, elegimos un paso t al azar, y generamos ruido ϵ.

Calculamos x_t usando la fórmula directa. Luego, le pedimos a la red ϵ_θ que adivine qué ruido ϵ usamos.

La función de pérdida ($\mathcal{L}_{simple}$) es simplemente el Mean Squared Error (MSE) entre el ruido real y el ruido predicho por la red:

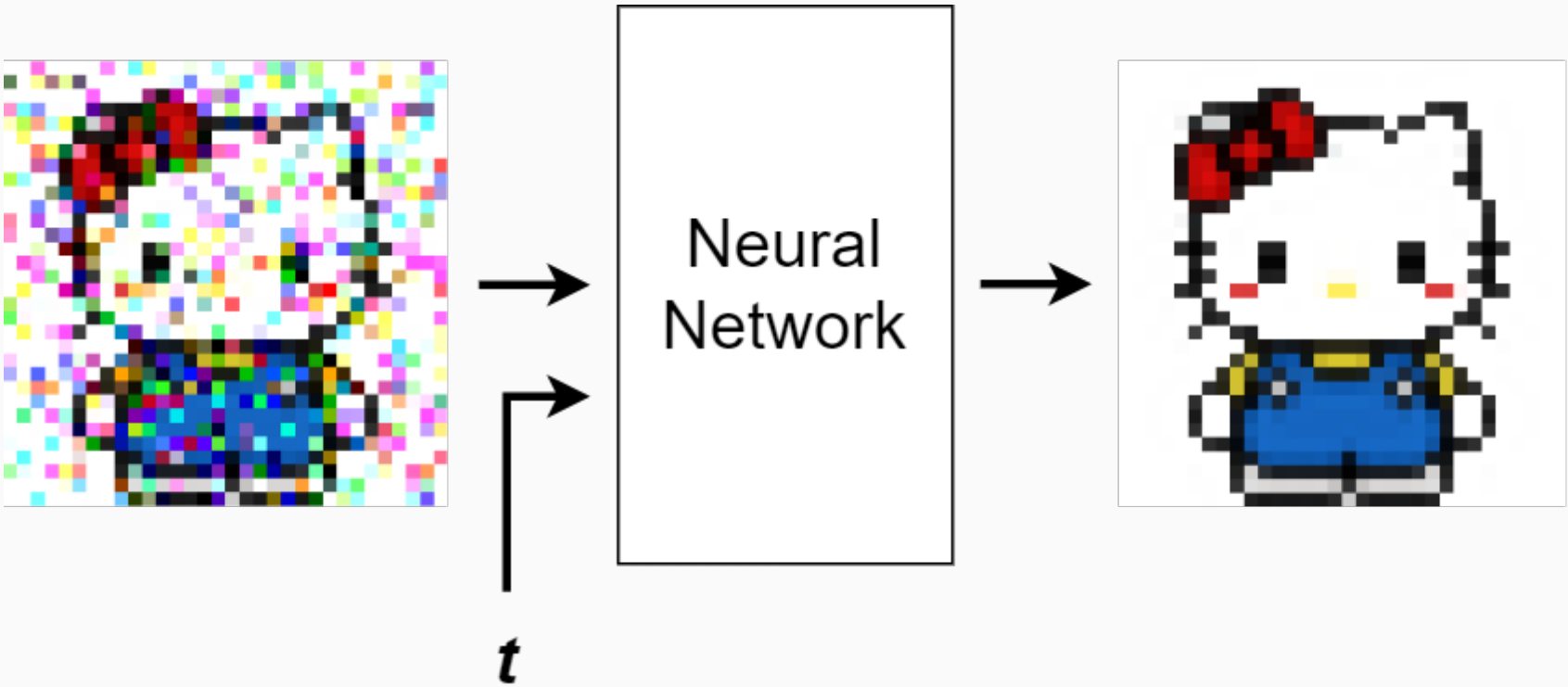
$$\mathcal{L}_{simple} = \mathbb{E}_{t,x_0,\epsilon}\left[||\epsilon - \epsilon_\theta(x_t, t)||^2\right] \tag{8}$$

La red aprende a separar la señal (imagen) del ruido en cualquier nivel de degradación t.

Arquitectura

Para poder realizar el paso de inverse diffusion utilizaremos redes neuronales, que serán capaces de calcular el ruido de una imagen de entrada.

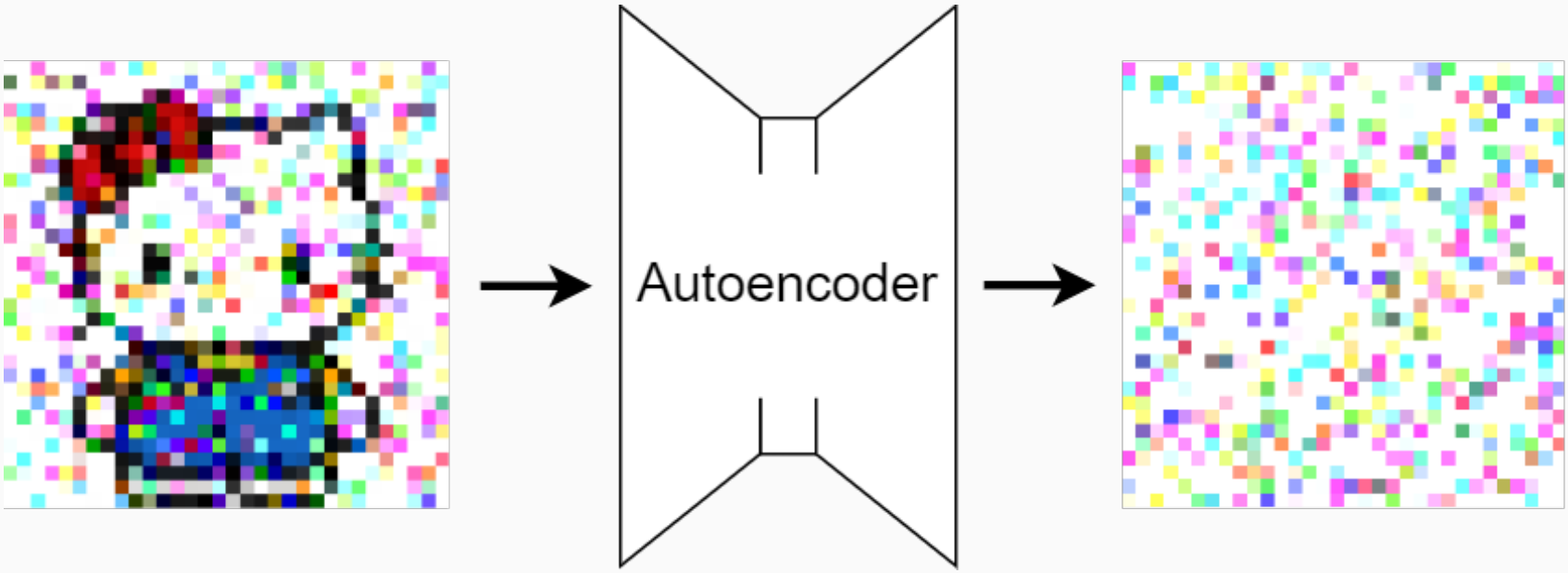
Dichas redes deberán calcular el ruido presente en las imágenes que se le presenten como entrada. Para ello es necesario que la red conozca el instante t en el que se encuentra.



Autoencoder para obtener el ruido

La solución más común para este tipo de problemas es el uso de Autoencoders. Como se ha visto anteriormente estos modelos generan como salida datos de la misma dimensionalidad que los de entrada.

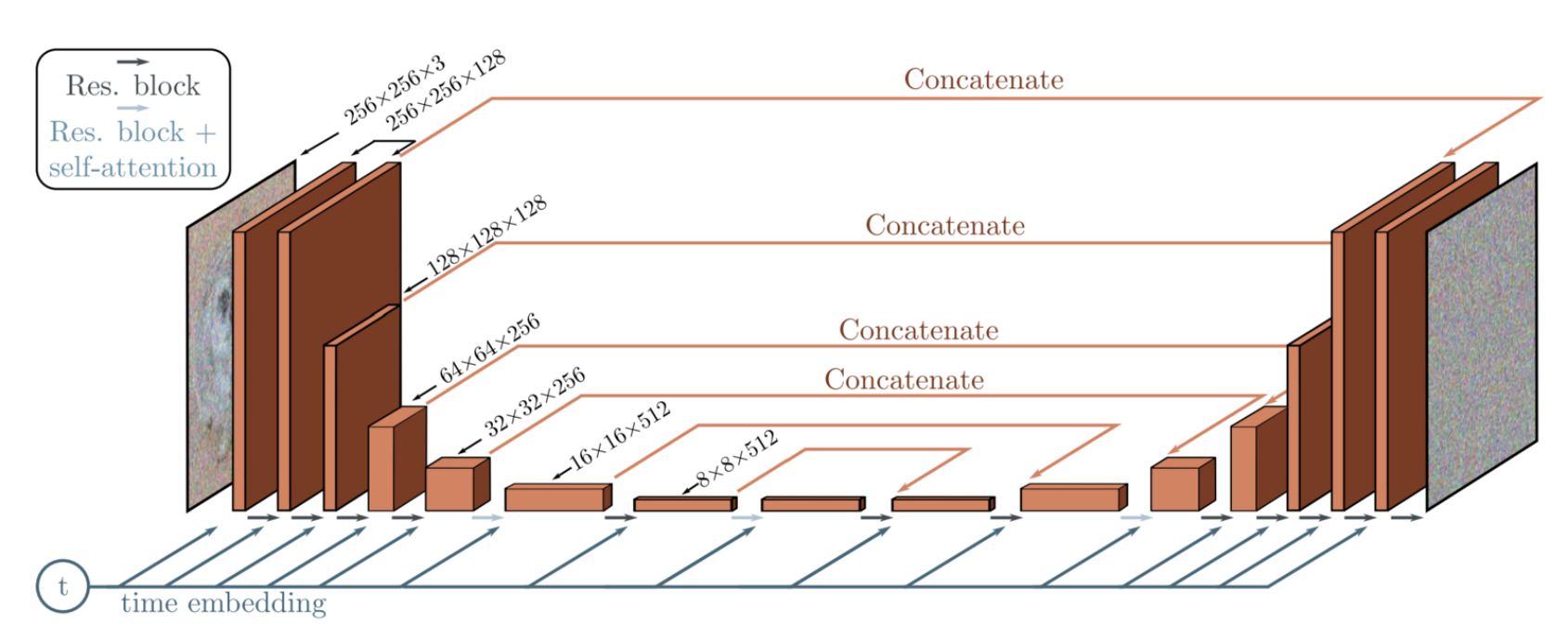
La función del Autoencoder será obtener cuál es el ruido de la imagen.



U-Net

La arquitectura más habitual para los diffusion models es la U-Net, por sus propiedades a la hora de recuperar información manteniendo la definición de las imágenes.

Aunque cabe destacar que dependiendo del conjunto de datos que se esté utilizando esto puede cambiar. E.g. diseñar un modelo de difusión para generar música nueva.



[7]

Algoritmo DDPM

El algoritmo Denoising Diffusion Probabilistic Model (DDPM), propuesto en el año 2020 [6], es el causante de la popularidad que han ganado los diffusion models los últimos años.

Este algoritmo permite eliminar el ruido de una imagen para recuperarla a su estado original.

Una de las características de DDPM es que añade cierta cantidad de ruido tras haberlo eliminado. Esto se hace para que la imagen de salida siga la misma distribución de datos del instante t.

Algoritmo DDPM

Añadiendo cierto ruido tras cada paso de la difusión inversa, se consigue aumentar la diversidad en las imágenes generadas, evitando el colapso modal.



[8]

Algoritmo DDPM: Generación de imágenes

Para generar una nueva imagen, partimos de ruido puro $x_T \sim \mathcal{N}(0,1)$ y aplicamos iterativamente la ecuación de muestreo inverso hasta llegar a x_0:

$$x_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)}_{\text{Paso hacia la imagen sin ruido}} + \underbrace{\sigma_t z}_{\text{Inyección de ruido}} \tag{9}$$

Donde $z \sim \mathcal{N}(0,1)$ si $t > 1$, y $z = 0$ si $t = 1$.

Desgranando la ecuación DDPM

- **Paso hacia la imagen:** Usamos la predicción de nuestra red (ϵ_θ) para restar el ruido estimado de x_t, escalado por las constantes α_t.
- **Inyección de ruido ($\sigma_t z$):** Al igual que en las Langevin Dynamics[3], se inyecta una pequeña cantidad de ruido nuevo en cada paso.

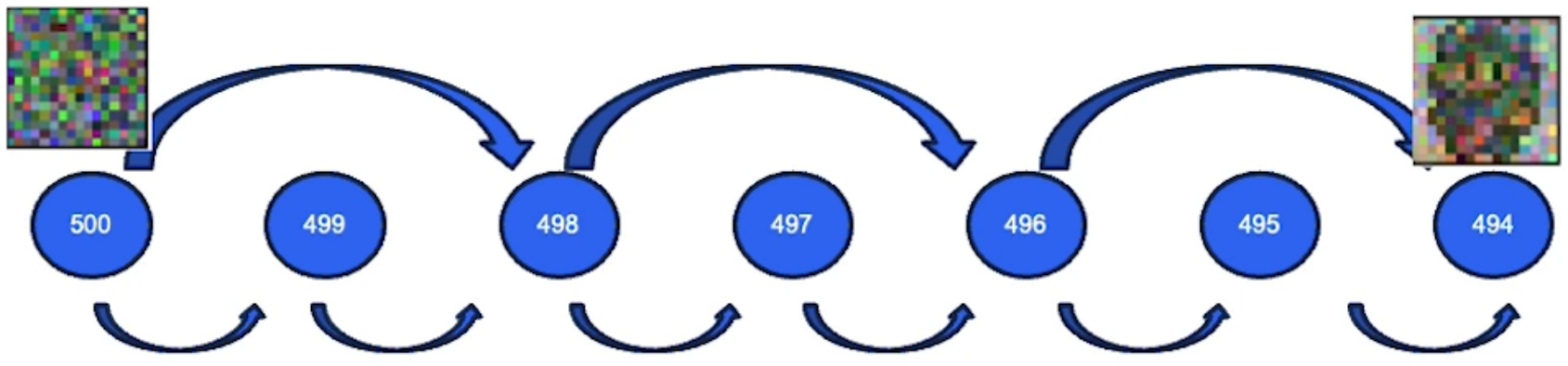
¿Por qué añadir ruido tras haberlo quitado? Esta aleatoriedad en cada paso evita que el modelo colapse en un único resultado promedio y garantiza la diversidad en las imágenes generadas.

[3] Vale, quizás meternos en modelado matemático para física es demasiado, pero simplemente hay que tener en cuenta que todo tiene su origen (y motivo).

Algoritmo DDIM

El algoritmo Denoising Diffusion Implicit Model (DDIM) [9] es una alternativa a DDPM más rápida y eficiente.

Se basa en la idea de saltar varios pasos de la inverse difussion de una única vez.



[8]

Algoritmo DDIM

La principal característica que permite a DDIM saltar pasos del proceso es que no es un modelo probabilístico, si no determinista.

No entraremos en profundidad, pero su funcionamiento se basa en el uso de ecuaciones diferenciales ordinarias para definir la cadena de Markov. Esto permite saltar pasos sin perder coherencia matemática.

De esta manera se elimina la aleatoriedad de las cadenas de Markov.

El algoritmo DDIM llega a ser 10 veces más eficiente que el DDPM.

Condicionamiento de la salida

Cómo elegir lo que se genera

A veces se desea tener cierto control sobre aquello que está generando nuestro modelo. Para ello tenemos que ser capaces de añadir cierta información extra a la red de neuronas.

Esta información extra se usará durante el entrenamiento para condicionar a la red a aprender la distribución de datos de las etiquetas que la condicionen.

De esta manera, cuando se quiera generar nueva información, se usará esas etiquetas para generar lo que se quiera.

Context embedding

Esto se realiza a través del conocido como context embedding. Este embedding se añade después de cada capa de la misma manera que el tiempo t.

Una de las ventajas de los embeddings es que puedes combinar información. Por ejemplo de la siguiente manera:



Gato

[1,0,0]

+



Dibujo

[0,0,1]

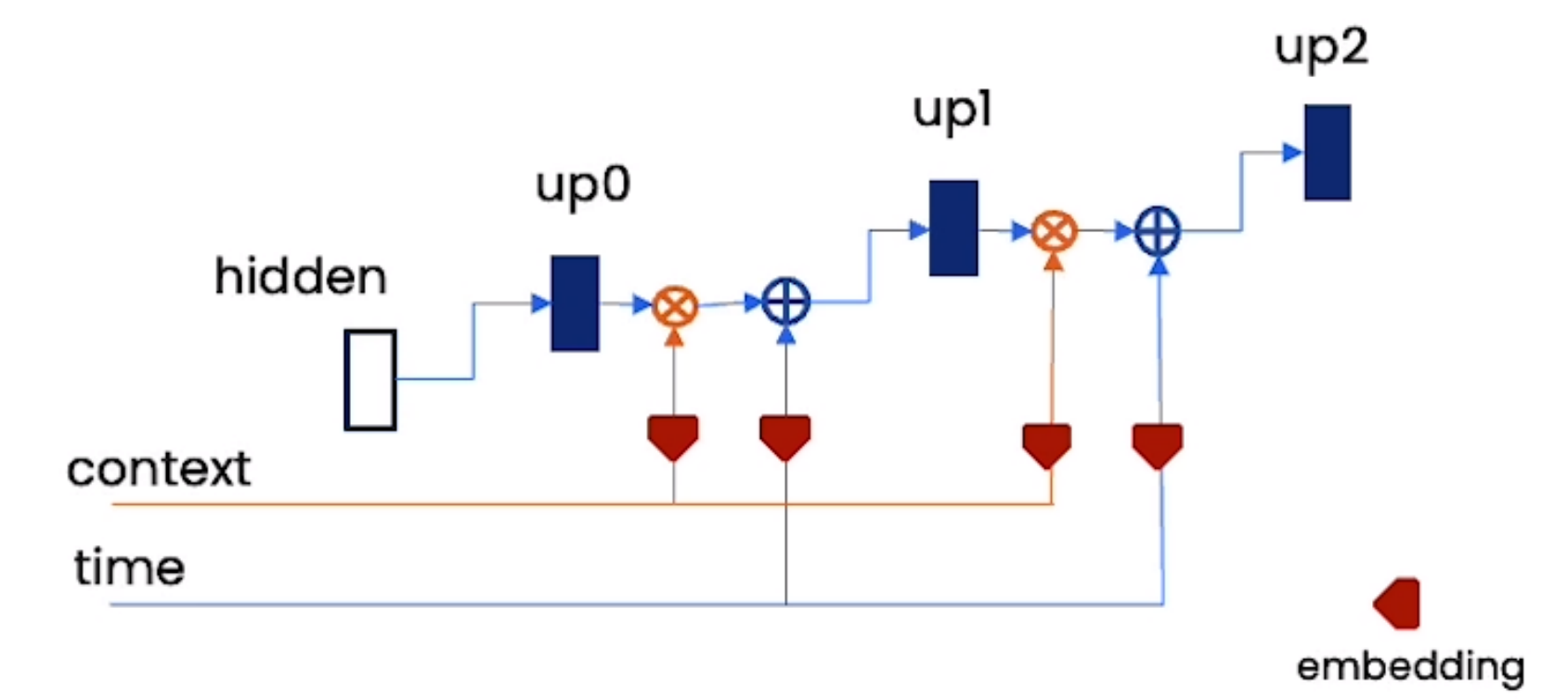
=



[1,0,1]

Context embedding

Este embedding es alimentado a la red después de cada bloque convolucional. De esta manera se condiciona a la red a que aprenda el contexto de la imagen.



[8]

Combinando con transformers

Si este embedding añadido se realiza con transformers, entonces somos capaces, por ejemplo, de controlar la salida a través de texto[4].

Este tipo de soluciones son las que hoy en dia se utilizan en DALL-E o Stable Diffusion.

[4] De hecho es lo más útil, sobran decir la cantidad de aplicaciones de esto hoy en día. Por ello a partir de aquí se habla de *texto*, pero es sustituible por *contexto*

El problema del condicionamiento

Aunque los transformers nos permiten extraer un *embedding* del texto (c) y pasárselo a nuestra red, a menudo, el modelo tiende a ignorar parcialmente la condición c porque aprenderla es más sencillo ignorar el texto y generar imágenes genéricas.

Históricamente, se usaba un clasificador externo para *empujar* los gradientes hacia la clase deseada (Classifier Guidance[10]), pero esto era ineficiente y costoso computacionalmente.

Classifier-Free Guidance: Entrenamiento

Para solucionar esto, en 2022 se popularizó el Classifier-Free Guidance (CFG)[11].

La idea es entrenar un único modelo de difusión que sea capaz de actuar tanto de forma condicionada (con texto) como incondicional (sin texto).

Durante el entrenamiento, se introduce un dropout de la condición: con una probabilidad pequeña (e.g. 10% de las veces), se sustituye el *embedding* de texto c por un token vacío o nulo ($\emptyset$).

- La red aprende a predecir el ruido con texto: $\epsilon_\theta(x_t, c)$
- La red aprende a predecir el ruido sin texto: $\epsilon_\theta(x_t, \emptyset)$

Classifier-Free Guidance: Inferencia

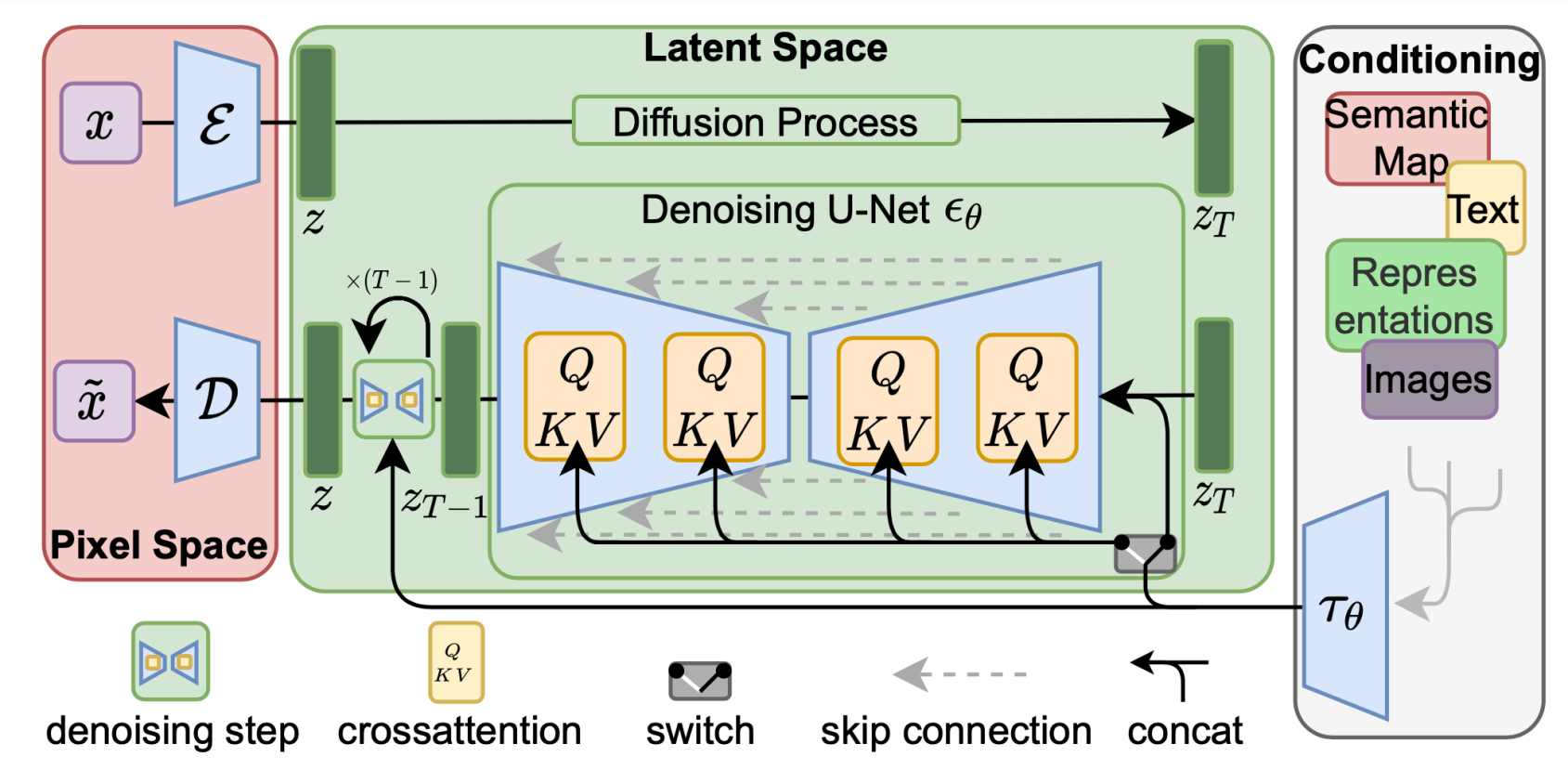
La verdadera magia ocurre en la inferencia. Para generar una imagen que siga estrictamente el texto, el modelo evalúa ambas condiciones y calcula la diferencia.

Usamos un parámetro w (Guidance Scale) para extrapolar el vector de ruido final $\tilde{\epsilon}_\theta$:

$$\tilde{\epsilon}_\theta = \epsilon_\theta(x_t, \emptyset) + w \cdot \Big(\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \emptyset)\Big) \tag{10}$$

- **El término entre paréntesis:** Es el vector que apunta desde una "imagen genérica" hacia la "imagen descrita por el texto".
- **Si $w = 1$:** Usamos el modelo condicionado normal.
- **Si $w > 1$ (ej. $w = 7$):** Forzamos al modelo a ir mucho más lejos en la dirección del texto, aumentando drásticamente la fidelidad al *prompt* a cambio de perder un poco de diversidad.

Resumen diffusion models



[12]

Diffusion models avanzados

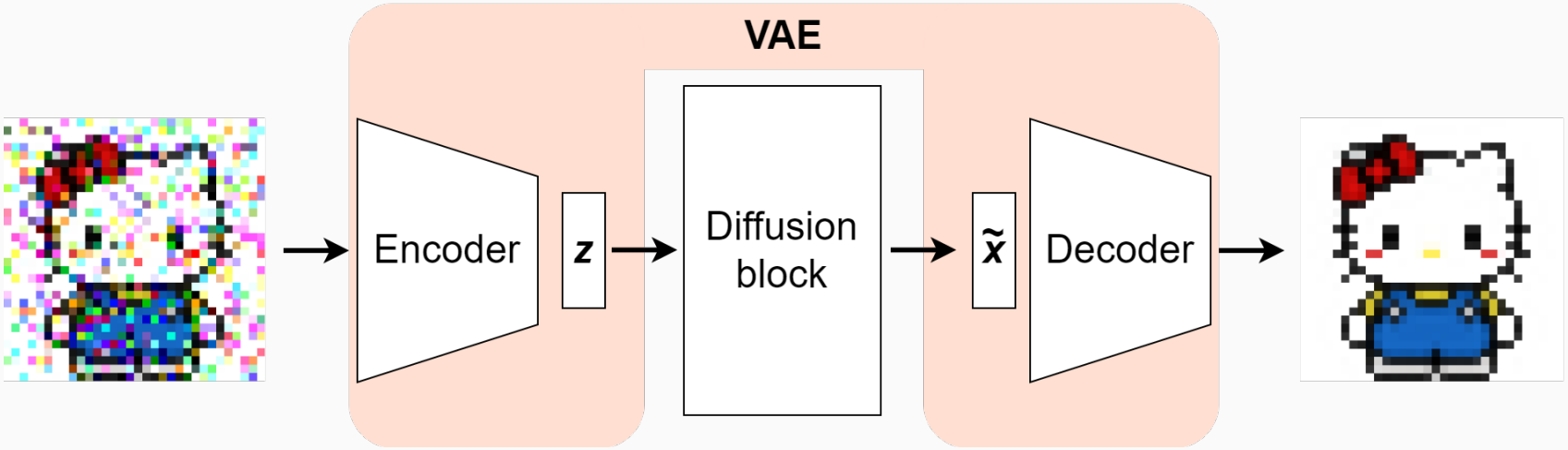
Latent Diffusion Model (Stable Diffusion)

Trabajar directamente sobre el espacio de píxeles de imágenes de alta resolución es computacionalmente muy costoso. La arquitectura Latent Diffusion Model (LDM) [12] propone:

- Un VAE codifica la imagen x a un espacio latente mucho más pequeño[5] $z = E(x)$. Al final, un decodificador reconstruye la imagen $\tilde{x} = D(z)$.
- **Compresión Semántica (Diffusion):** El proceso de difusión (forward y reverse) no se hace con píxeles, sino iterando exclusivamente dentro de ese espacio latente z.

[5] E.g. de $512 \times 512 \times 3$ a $64 \times 64 \times 4$

Latent Diffusion Model

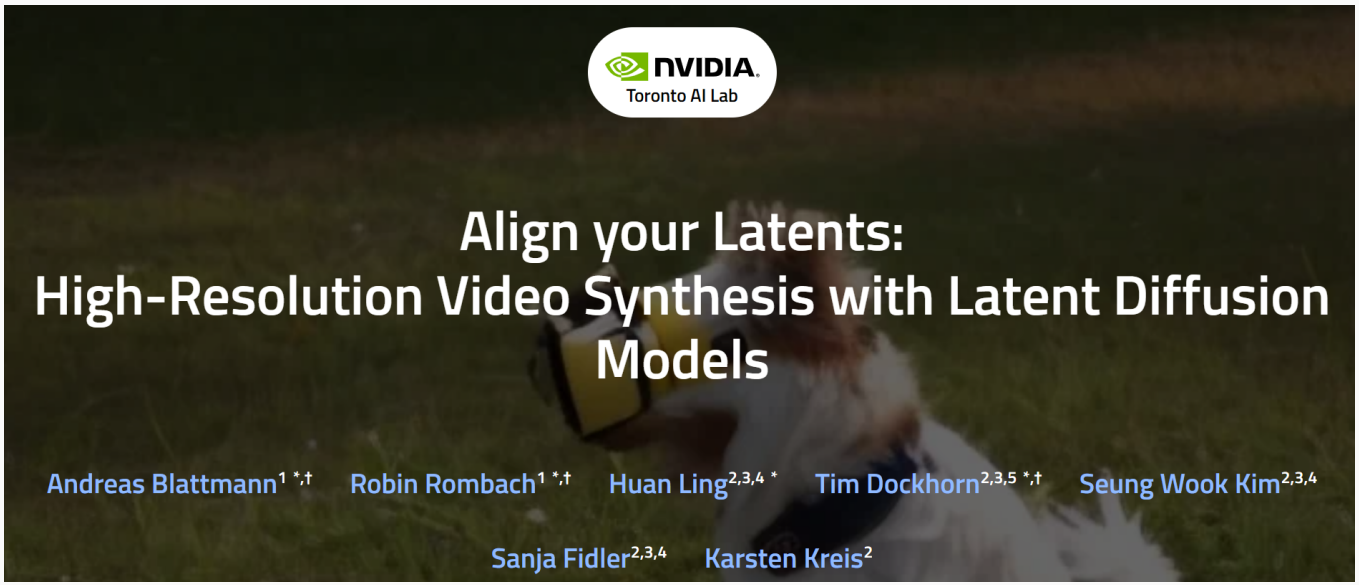


Esto reduce drásticamente el coste computacional y es el secreto del éxito de los nuevos modelos de generación.

Generación de video: Align your latents

A parte de generar imágenes, existen alternativas capaces de generar vídeo. Uno de los trabajos más importantes fue publicado por Andreas Blattmann et al., de NVIDIA [13] en el año 2023.

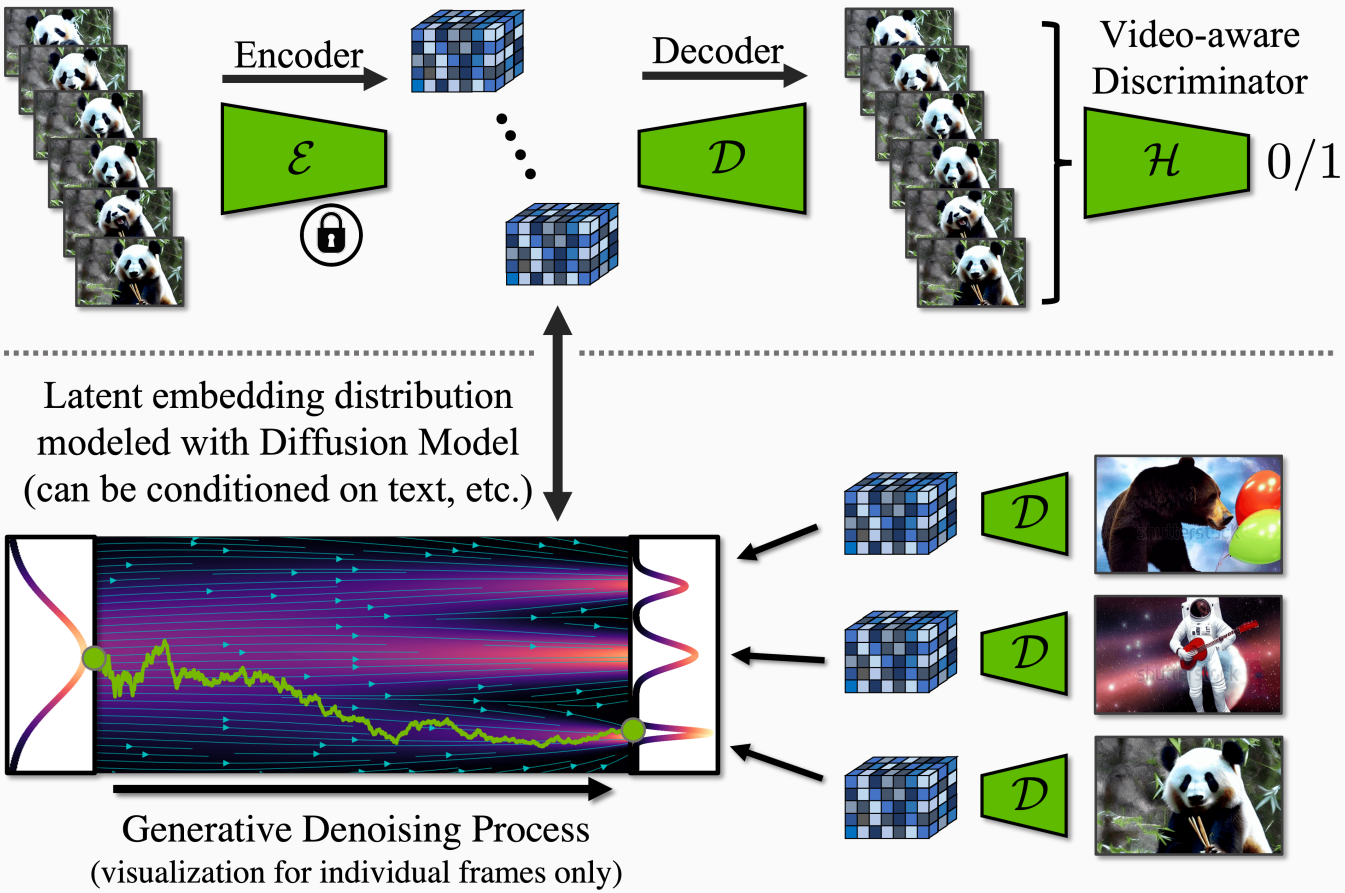
Este trabajo usa la arquitectura LDM introduciendo un batch de imágenes que corresponden con un vídeo de 4,7 segundos.



Ejemplos del resultado en la Web de NVIDIA

Align your latents

En la arquitectura propuesta se usan ideas también de GAN, como el uso de un discriminador para que los videos mantengan fotorealismo.



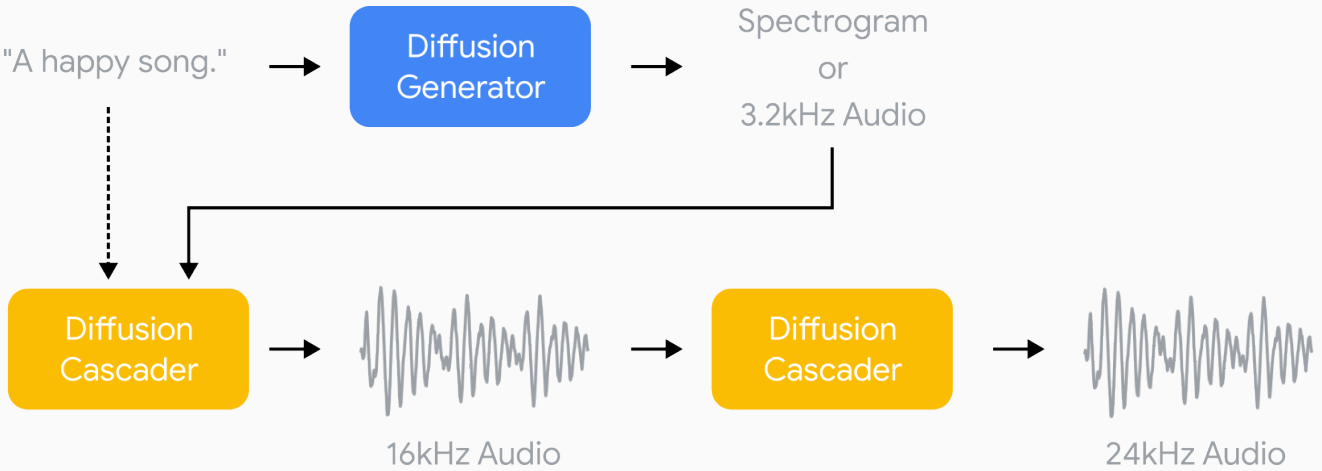
[13]

Generación de música: Noise2Music

El trabajo de Noise2Music [14] propone el uso de modelos de difusión para generar música a partir de un texto.

Para ello se proponen 2 componentes principales:

- El diffusion generator se encarga de generar la base de la música a baja calidad a partir de la query de texto.
- Una serie de diffusion cascader que se encargan de mejorar la calidad iterativamente.



[14]

Noise2Music

En el trabajo original proponen dos alternativas para tratar el audio:

- Ondas sonoras: Tratan la música como una serie temporal. Para ello se usa una 1D U-Net en la que las convoluciones observan una única dimensión.
- Espectrograma: Una alternativa bastante común para tratar con audio es transformarlo a imagen usando su espectograma.

Referencias i

[1] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli.
Deep unsupervised learning using nonequilibrium thermodynamics.
In *International conference on machine learning*, pages 2256–2265. PMLR, 2015.

[2] A Vahdat and Karsten Kreis.
Improving diffusion models as an alternative to gans, part 1. nvidia technical blog.
NVIDIA Developer, 2022.

Referencias ii

[3] J. Rafid Siddiqui (towards data science).
Diffusion model basic scheme.
https://towardsdatascience.com/diffusion-models-made-easy-8414298ce4da.
[Online; accessed August, 2023].

[4] Ayan Das.
Forward diffusion image.
https://ayandas.me/blog-tut/2021/12/04/diffusion-prob-models.html.
[Online; accessed August, 2023].

[5] Alexander Quinn Nichol and Prafulla Dhariwal.
Improved denoising diffusion probabilistic models.
In *International conference on machine learning*, pages 8162–8171. PMLR, 2021.

Referencias iii

[6] Jonathan Ho, Ajay Jain, and Pieter Abbeel.
Denoising diffusion probabilistic models.
Advances in neural information processing systems, 33:6840–6851, 2020.

[7] Vaibhav Singh (LearnOpenCV).
U-net architecture image.
https://learnopencv.com/denoising-diffusion-probabilistic-models/.
[Online; accessed August, 2023].

[8] Sharon Zhou.
How diffusion models work.
https://learn.deeplearning.ai/diffusion-models.
[Online; accessed August, 2023].

Referencias iv

[9] Jiaming Song, Chenlin Meng, and Stefano Ermon.
Denoising diffusion implicit models.
arXiv preprint arXiv:2010.02502, 2020.

[10] Prafulla Dhariwal and Alexander Nichol.
Diffusion models beat GANs on image synthesis.
In *Advances in Neural Information Processing Systems*, volume 34, pages 8780–8794, 2021.

[11] Jonathan Ho and Tim Salimans.
Classifier-free diffusion guidance.
arXiv preprint arXiv:2207.12598, 2022.

Referencias v

[12] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer.
High-resolution image synthesis with latent diffusion models.
In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[13] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis.
Align your latents: High-resolution video synthesis with latent diffusion models.
In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22563–22575, 2023.

Referencias vi

[14] Qingqing Huang, Daniel S Park, Tao Wang, Timo I Denk, Andy Ly, Nanxin Chen, Zhengdong Zhang, Zhishuai Zhang, Jiahui Yu, Christian Frank, et al.
Noise2music: Text-conditioned music generation with diffusion models.
arXiv preprint arXiv:2302.03917, 2023.

Citación

Cita APA: Iglesias, G. and Talavera, E. (2026, noviembre 7). *Diffusion models* [Diapositivas de la asignatura Métodos Generativos]. GitHub. *https://guillermoih.github.io/M-todos-Generativos/Diffusion_Models.pdf*

Tema de las diapositivas "Metropolis" lincenciado bajo LPPL 1.3c

BibTeX

```
@slides{Diffusion_Models_MG_2026,
   author = {Iglesias, Guillermo and Talavera, Edgar},
   title = {Diffusion models [Diapositivas de la asignatura Métodos Generativos]},
   year = {2026},
   month = {11},
   howpublished={\url{https://guillermoih.github.io/M-todos-Generativos/Diffusion_Models.pdf}},
   note = {[Online; accessed 7 de noviembre de 2026]},
   organization={ETSISI. Universidad Politécnica de Madrid}
}
```